

MVVM (Model-View-ViewModel)

Model — View — ViewModel (MVVM) is the industry-recognized software architecture pattern that overcomes all drawbacks of MVP and MVC design patterns. MVVM suggests separating the data presentation logic (Views or UI) from the core business logic part of the application. **The ultimate goal of MVVM architecture is to make the view completely independent from the application logic.**

Layers of MVVM

- **Model:** This layer is responsible for the abstraction of the data sources. Model and ViewModel work together to get and save the data.
- **View:** The View represents the user interface of the application and holds limited, purely presentational logic that implements visual behavior. The View is completely agnostic to the business logic. In other words, the View is a “**dumb**” class that never contains data, nor manipulates it directly. It communicates with the ViewModel through data binding and is unaware of the Model.
- **ViewModel:** It exposes those data streams which are relevant to the View. Moreover, it serves as a link between the Model and the View. **It implements and exposes public properties and commands that the View uses by way of data binding. If any state changes occur, the ViewModel notifies the View through notification events.**

MVVM pattern has some similarities with the MVP (Model — View — Presenter) design pattern as the Presenter role is played by ViewModel. However, **the drawbacks of the MVP pattern have been solved by MVVM in the following ways:**

1. **ViewModel does not hold any kind of reference to the View.**
2. **Many-to-1 relationships exist between View and ViewModel.**
3. **No triggering methods to update the View.**

How Does MVVM Architecture Work?

The key to understanding MVVM architecture is understanding how the three key components in MVVM interact with each other. Since **the View only communicates with the ViewModel and the ViewModel only communicates with the Model.**

All user interaction occurs within the View, which is in charge of detecting the user's input (mouse clicks, keyboard input) and forwarding it to the ViewModel by way of data binding. Data binding can be implemented with callbacks or properties and constitutes the concrete link between the View and the ViewModel.

The ViewModel implements the properties and commands to which the view can be bound. These properties and commands define the functionality that the View can offer to the user, although how to display it is entirely up to the View. The ViewModel is also in charge of providing the View with data from the Model classes, which the View can consume. To accomplish this, the ViewModel can expose Model classes directly to the View, in which case the Model class would need to support data binding and change notification events.

The Models are classes that model the application's domain. Models encapsulate the application's data and business logic. They can be considered business objects that have absolutely no relation to the visual aspect of the application.

when to use **MVC**, **MVP**, or **MVVM**?

MVC (Model–View–Controller)

Best for

- Web apps & server-side frameworks where MVC is built-in (Rails, Django, ASP.NET MVC).
- Small to medium-sized projects.
- When UI updates are simple and you don't need complex binding logic.

Pros

- Simple to understand.
- Often supported natively by frameworks.
- Good for request/response-based flows (web pages).

Cons

- View & Controller can become tightly coupled.
- Not great for highly interactive UIs — manual wiring for data updates.

MVP (Model–View–Presenter)

Best for

- **Native mobile apps** (especially **pre–data binding Android** days) and desktop apps.
- When you need **testability** — Presenter can be tested without the View.
- When you want a **passive View** (no UI logic at all).

Pros

- Excellent separation of concerns.
- View is easy to swap (e.g., for different platforms).
- Presenter is platform-agnostic and testable.

Cons

- More boilerplate — Presenter has to know all View methods.
- Can get verbose in complex screens.

MVVM (Model–View–ViewModel)

Best for

- Modern **reactive/declarative UIs** where data binding or state management is available.
- **Android Jetpack (ViewModel + LiveData/StateFlow), SwiftUI, WPF, Angular, Vue, React** (with hooks/state).
- Apps with **frequent UI updates** or multiple UI components depending on the same data.

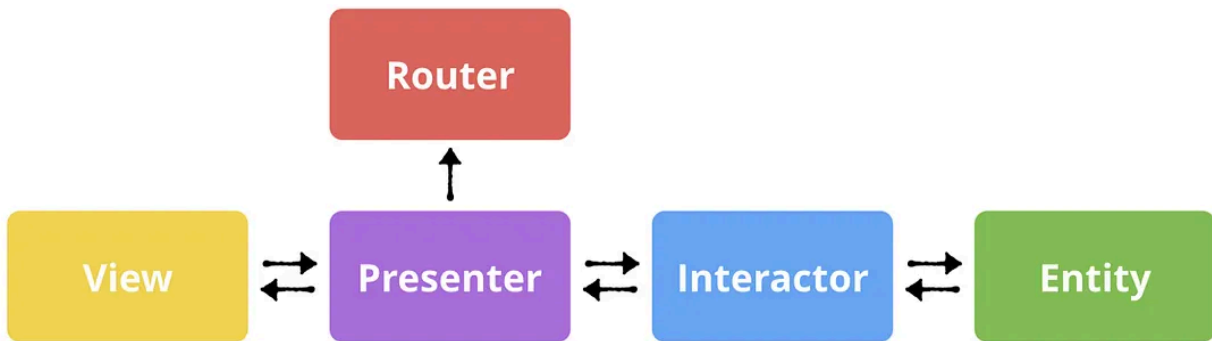
Pros

- Minimal boilerplate when paired with data binding.
- Very testable — ViewModel has no UI references.
- Automatic UI updates (via data binding or state observation).

Cons

- Data binding/debugging can be tricky.
- Overkill for very simple UIs.

VIPER



VIPER is an architectural pattern like MVC or MVVM, but it separates the code further by single responsibility. Apple-style MVC motivates developers to put all logic into a UIViewController subclass. VIPER, like MVVM before it, seeks to fix this problem.

Each of the letters in VIPER stand for a component of the architecture: View, Interactor, Presenter, Entity and Router.

The **View** is the user interface. This corresponds to a SwiftUI View. It is our ViewController and it's responsibility is showing data to user. Here we only use the data from the presenter. No other action is taken.

The **Interactor** is a class that mediates between the presenter and the data. It takes direction from the presenter. We can say that it acts as the ViewModel in the MVVM design pattern. It is the part of the application that we call Business Logic. UI operations are not performed here. Fetch, Update, API etc. operations take place here.

The **Presenter** is the “traffic cop” of the architecture, directing data between the view and interactor, taking user actions and calling to router to move the user between views. It is the bridge between View and Interactor. This layer should not contain UI or business logic operations.

An **Entity** represents application data. Data models related to the application are found here. This part cooperates only with Interactor.

The **Router** handles navigation between screens. This layer is the layer that allows us to determine when the application's pages are shown. Router helps to communicate other layers with each other. We describe our **Protocols** here.

This separation is borne out of “Uncle” Bob Martin’s Clean Architecture paradigm.

Protocols are important in this architecture. In Swift, a protocol is like a contract (**interface**):

- It declares methods/properties a class/struct must implement.
- In VIPER, each module defines protocols for communication between its layers.
- This keeps layers independent and decoupled → you can replace one part without breaking others.

There are 2 types of it, carrier protocols and main protocols.

InteractorToPresenterProtocol and PresenterToViewProtocol are the carrier protocols. These protocols help to carry the final data formed in the interactor from the presenter to the view.

ViewToPresenterProtocol and PresenterToInteractorProtocol are main protocols, so they includes not only functions but also properties. We use these protocols to send data from view to interactor through presenter.

There is no need to define variables in carrier protocols. They carry data with methods. However, in the main protocols, we need to trigger data by defining variables.